Week 1->1.2 Desing Patterns

A design pattern in programming is a reusable solution to a common problem that occurs during software design and development. It provides a structured approach to solving specific design or implementation issues, allowing developers to create more maintainable, flexible, and scalable code.

Design patterns capture best practices and proven solutions that have been refined over time by experienced developers. They provide a common language and set of concepts for discussing and communicating design decisions and patterns of interaction between classes or objects.

Design patterns can be categorized into three main types:

1. **Creational Patterns:** These patterns focus on object creation mechanisms, providing ways to create objects in a manner that is flexible, decoupled from their concrete implementations, and suited to the specific requirements of the application. Examples include the Singleton pattern, Factory pattern, and Builder pattern.

2. **Structural Patterns**: These patterns deal with the composition of classes and objects to form larger structures while keeping them flexible and efficient. They help ensure that classes and objects can work together effectively to achieve a common goal. Examples include the Adapter pattern, Decorator pattern, and Composite pattern.

   Structural design patterns are design patterns that deal with how classes and objects are connected or composed to form larger structures — while keeping things flexible and easy to maintain.

3. **Behavioral Patterns**: These patterns focus on the interactions and communication between objects and classes. They provide solutions for effectively managing the flow of control, responsibilities, and behavior between objects. Examples include the Observer pattern, Strategy pattern, and Command pattern.

Singleton Design Pattern

The Singleton design pattern is a creational pattern that ensures the creation of only one instance of a class and provides a global point of access to it.

Example:

```
public class Singleton {
  // Step 1: Create a private static variable to hold the one instance
  private static Singleton instance;

  // Step 2: Make the constructor private so no one can create objects directly
  private Singleton() {
    System.out.println("Singleton instance created.");
  }

  // Step 3: Provide a public static method to get the instance
  public static Singleton getInstance() {
    if (instance == null) {
      instance = new Singleton();  // Create instance only once
    }
    return instance;
  }

  public void showMessage() {
    System.out.println("Hello from Singleton!");
  }
}
```

Using the Singleton:

```
public class Main {
    public static void main(String[] args) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();


        s1.showMessage();


        // Check if both are same
        System.out.println(s1 == s2);  // Output: true
    }
}
```

Factory Design Pattern:

The Factory Pattern is a creational design pattern that helps in creating objects without exposing the actual object creation logic to the user (client).

Instead of creating objects directly with the new keyword, you ask a factory class to do it for you. This factory class has a method (usually called a factory method) that decides which class object to create and then returns it.

This pattern is especially useful when you have a common interface or parent class, and many subclasses (or implementations) that you can create based on different needs or input.

So, the Factory Pattern hides the complexity of object creation and centralizes it in one place, making your code easier to extend, maintain, and reuse.

The Factory Method Pattern is a design pattern where you delegate object creation to a method (the factory), instead of creating objects directly using the new keyword.

Step 1: Create a Food interface

```
public interface Food {
   void prepare();
}
```

Step 2: Implement classes: Pizza, Burger, Pasta

```
public class Pizza implements Food {
   public void prepare() {
      System.out.println("Preparing Pizza...");
   }
}

public class Burger implements Food {
   public void prepare() {
      System.out.println("Preparing Burger...");
   }
}

public class Pasta implements Food {
   public void prepare() {
      System.out.println("Preparing Pasta...");
   }
}
```

Step 3: Factory class to create objects

```
public class FoodFactory {
```

```
    public static Food getFood(String type) {
        if(type.equalsIgnoreCase("Pizza"))
            return new Pizza();
        else if(type.equalsIgnoreCase("Burger"))
            return new Burger();
        else if(type.equalsIgnoreCase("Pasta"))
            return new Pasta();
        else
            return null;
    }
}
```

Step 4: Main class

```
public class Main {
    public static void main(String[] args) {
        String userChoice = "Burger";

        Food foodItem = FoodFactory.getFood(userChoice);

        if(foodItem != null)
            foodItem.prepare();
        else
            System.out.println("Sorry, we don't serve that!");
    }
}
```

Problems:

- Every time you add a new item (e.g. Sandwich), you have to **edit the main() method**.
- Code becomes **long, messy, and hard to maintain**.
- **Tight coupling**: main() knows all classes (Pizza, Burger, Pasta).

Builder Pattern

The **Builder Pattern** lets you create a big or complex object **step-by-step**, instead of using a long constructor with too many parameters. You don't call new with many arguments — instead, you use a **Builder class** that guides the creation process clearly.

```
public class Pizza {
    private String size;
    private boolean cheese;
    private boolean pepperoni;

    // Constructor - only called by the builder
    private Pizza(PizzaBuilder builder) {
        this.size = builder.size;
        this.cheese = builder.cheese;
        this.pepperoni = builder.pepperoni;
    }

    public void display() {
        System.out.println("Pizza size: " + size);
        System.out.println("Cheese: " + cheese);
        System.out.println("Pepperoni: " + pepperoni);
    }
```

```
    // Static Builder class
    public static class PizzaBuilder {
        private String size;
        private boolean cheese;
        private boolean pepperoni;

        public PizzaBuilder(String size) {
            this.size = size;
        }

        public PizzaBuilder addCheese() {
            this.cheese = true;
            return this;
        }

        public PizzaBuilder addPepperoni() {
            this.pepperoni = true;
            return this;
        }

        public Pizza build() {
            return new Pizza(this);
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Pizza myPizza = new Pizza.PizzaBuilder("Large")
                        .addCheese()
                        .addPepperoni()
                        .build();

        myPizza.display();
    }
}
```

Pizza size: Large

Cheese: true

Pepperoni: true

Adapter Design Pattern

The Adapter design pattern is a structural pattern that allows incompatible classes to work together by converting the interface of one class into another interface that clients expect. It acts as a bridge between two incompatible interfaces. The Adapter pattern is useful when you want to reuse existing code without making significant changes to it.

Example:

Scenario: Electrical Socket and Plug

Problem:

- You have a **TwoPinPlug** (like an Indian plug).
- Your new socket only accepts **ThreePinPlug**.
- But you can't directly connect a two-pin plug to a three-pin socket.

Solution: Use an **adapter** that allows a TwoPinPlug to fit into a ThreePinSocket.

Step 1: Target Interface – What the client expects

```
public interface ThreePinPlug {
   void connectThreePin();
}
```

Step 2: Adaptee – Already existing incompatible class

```
public class TwoPinPlug {
   public void connectTwoPin() {
      System.out.println("Connected using Two Pin Plug.");
   }
}
```

Step 3: Adapter – Converts TwoPinPlug to work with ThreePinPlug

```
public class PlugAdapter implements ThreePinPlug {
   private TwoPinPlug twoPinPlug;

   public PlugAdapter(TwoPinPlug twoPinPlug) {
      this.twoPinPlug = twoPinPlug;
   }

   public void connectThreePin() {
      System.out.print("Adapter in action → ");
      twoPinPlug.connectTwoPin();
   }
}
```

Step 4: Client code using the Adapter

```
public class Main {
    public static void main(String[] args) {
        // Old plug
        TwoPinPlug oldPlug = new TwoPinPlug();

        // Use adapter to connect old plug to new socket
        ThreePinPlug adapter = new PlugAdapter(oldPlug);

        adapter.connectThreePin();  // Now compatible
    }
}
```

✅ **Output:**

Adapter in action → Connected using Two Pin Plug.

Decorator Design Pattern

The Decorator Pattern is a structural design pattern that lets you add new behavior to objects dynamically at runtime, without modifying their code.

The Decorator Pattern is a structural design pattern that allows behavior to be added to an individual object dynamically, without affecting the behavior of other objects from the same class. It provides a flexible alternative to subclassing for extending the functionality of an object at runtime

Example:

Imagine you're ordering a basic coffee ☕ .

You can **add extra things like milk, sugar, whipped cream** — each one **adds something to the original coffee**.

But you're **not changing the original coffee class**. You're just **wrapping** it with new features.

That’s exactly what the **Decorator Pattern** does in programming.

1. Step: Create the base interface

```
public interface Coffee {
   String getDescription();
   double getCost();
}
```

---

2. Step: Concrete implementation of Coffee

```
public class SimpleCoffee implements Coffee {
   public String getDescription() {
      return "Simple Coffee";
   }

   public double getCost() {
      return 5.0;
   }
}
```

---

3. Step: Abstract Decorator Class

```
public abstract class CoffeeDecorator implements Coffee {
   protected Coffee decoratedCoffee;

   public CoffeeDecorator(Coffee c) {
      this.decoratedCoffee = c;
   }
```

```
    public String getDescription() {
        return decoratedCoffee.getDescription();
    }

    public double getCost() {
        return decoratedCoffee.getCost();
    }
}
```

---

4. Step: Add Concrete Decorators (Milk, Sugar)

```
public class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee c) {
        super(c);
    }

    public String getDescription() {
        return super.getDescription() + ", Milk";
    }

    public double getCost() {
        return super.getCost() + 1.5;
    }
}

public class SugarDecorator extends CoffeeDecorator {
```

```
    public SugarDecorator(Coffee c) {
        super(c);
    }

    public String getDescription() {
        return super.getDescription() + ", Sugar";
    }

    public double getCost() {
        return super.getCost() + 0.5;
    }
}
```

---

5. Step: Test in main class

```
public class Main {
    public static void main(String[] args) {
        Coffee coffee = new SimpleCoffee();
        System.out.println(coffee.getDescription() + " $" + coffee.getCost());

        coffee = new MilkDecorator(coffee);
        System.out.println(coffee.getDescription() + " $" + coffee.getCost());

        coffee = new SugarDecorator(coffee);
        System.out.println(coffee.getDescription() + " $" + coffee.getCost());
    }
}
```

✅ **Output:**

Simple Coffee $5.0

Simple Coffee, Milk $6.5

Simple Coffee, Milk, Sugar $7.0

Observer Design Pattern

The Observer design pattern is a behavioral pattern that establishes a one-to-many dependency between objects, such that when one object changes its state, all its dependents are notified and updated automatically. It is useful in scenarios where multiple objects need to be notified about changes in another object's state.

Structure of Observer Pattern

1. **Subject (Publisher)**
    - Maintains a list of observers
    - Provides methods to add/remove observers
    - Notifies them when something changes
2. **Observer (Subscriber)**
    - Interface/class with an update() method
    - Gets called when subject state changes
3. **ConcreteSubject and ConcreteObserver**
    - Real implementations of the pattern

✅ **Java Example: News Agency & Subscribers**

Step 1: Define Observer interface

public interface Observer {

```
    void update(String news);
}
```

Step 2: Create Subject interface

```
public interface Subject {
    void registerObserver(Observer o);
    void removeObserver(Observer o);
    void notifyObservers();
}
```

Step 3: Concrete Subject — NewsAgency

```
import java.util.*;
public class NewsAgency implements Subject {
    private List<Observer> observers = new ArrayList<>();
    private String news;

    public void setNews(String news) {
        this.news = news;
        notifyObservers(); // Notify all observers when news changes
    }

    public void registerObserver(Observer o) {
        observers.add(o);
    }

    public void removeObserver(Observer o) {
```

```
        observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer o : observers) {
            o.update(news);
        }
    }
}
```

Step 4: Concrete Observers — Subscribers

```
public class EmailSubscriber implements Observer {
    private String name;

    public EmailSubscriber(String name) {
        this.name = name;
    }

    public void update(String news) {
        System.out.println(name + " received news update via Email: " + news);
    }
}

public class SMSSubscriber implements Observer {
    private String name;
```

```
    public SMSSubscriber(String name) {
        this.name = name;
    }

    public void update(String news) {
        System.out.println(name + " received news update via SMS: " + news);
    }
}
```

Step 5: Test it with Main

```
public class Main {
    public static void main(String[] args) {
        NewsAgency agency = new NewsAgency();

        Observer emailUser = new EmailSubscriber("Alice");
        Observer smsUser = new SMSSubscriber("Bob");

        agency.registerObserver(emailUser);
        agency.registerObserver(smsUser);

        agency.setNews("Breaking News: Observer Pattern Simplified!");
    }
}
```